Mini Project: Cohort 21: Affordable Housing Price Estimator

Date : 28.02.2026

Model building and Evaluation

1.Importing Libraries:

At the start of the project, we imported all the required Python libraries. These libraries make our work easier by providing ready-made functions for handling data, performing calculations, drawing graphs, and building machine learning models. Without these libraries, implementing each task from scratch would be very difficult and time-consuming.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

2. Loading the Dataset

After importing the libraries, we loaded the housing dataset into the program. This dataset contains details of different houses such as their size, distance to the nearest MRT station, and price. Loading the dataset allows us to view the data and begin the analysis process.

```
df = pd.read_csv('content/Bengaluru_House_Data_Cleaned (1).csv')
print("Shape:", df.shape)
print(df.head())
```

3. Exploratory Data Analysis (EDA)

Next, we explored the dataset to understand its structure and contents. We checked statistical information like average values, minimum and maximum values, and also looked for missing data. This step is important because it helps us know whether the data needs cleaning before using it for training the model.

```
print(df.describe())
```

```
print(df.isnull().sum())
```

4. Selecting Important Features

From the dataset, we selected the features that have the most impact on house prices. Factors like total square footage and distance to the nearest MRT station are very important in determining property value, so we focused on these variables for further analysis.

```
cols = ['total_sqft', 'Distance_to_nearest_MRT_station_m', 'price']
print("\nSkewness:")
print(df[cols].skew())
```

5. Data Visualization

We plotted graphs to see how the data is distributed. Visualization helps us understand patterns in the data easily compared to just looking at numbers. It also helps us detect whether the data is balanced or skewed.

```
fig, axes = plt.subplots(1, 3, figsize=(14, 4))
for i, col in enumerate(cols):
    axes[i].hist(df[col], bins=40, color='steelblue', edgecolor='black')
    axes[i].set_title(f'{col}\nSkewness: {df[col].skew():.2f}')
    axes[i].set_xlabel(col)
plt.suptitle('Skewness Check - Before Scaling', fontsize=13)
plt.tight_layout()
plt.show()
```

6. Outlier Detection

We checked for outliers using boxplots. Outliers are unusual values that are much higher or lower than the rest of the data. In housing datasets, these may represent luxury houses or extremely small properties. Detecting them is important because they can affect the model's accuracy.

```
fig, axes = plt.subplots(1, 3, figsize=(14, 4))
for i, col in enumerate(cols):
    axes[i].boxplot(df[col], patch_artist=True,
                    boxprops=dict(facecolor='lightblue'))
    axes[i].set_title(f'Outliers: {col}')
plt.suptitle('Outlier Detection - Boxplot', fontsize=13)
plt.tight_layout()
plt.show()
```

7. Counting Outliers Using IQR

We used the Interquartile Range (IQR) method to measure outliers more accurately. This method identifies values that fall far outside the normal range of the data.

```
print("\n─ Outlier Count using IQR ─")
for col in cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    outliers = df[(df[col] < lower) | (df[col] > upper)]
    print(f"{col}: {len(outliers)} outliers "
          f"(lower bound: {lower:.2f}, upper bound: {upper:.2f})")
```

8. Comparing Scaling Techniques

Before training the model, we tested different scaling methods. Scaling is important because features with very large values can dominate the model. We chose RobustScaler because it works well even when outliers are present.

```
from sklearn.preprocessing import StandardScaler, RobustScaler


sample = df[['total_sqft']].copy()

std_scaled   = StandardScaler().fit_transform(sample)
robust_scaled = RobustScaler().fit_transform(sample)
```

9. Removing Extreme Values

We removed extremely high or low values from the dataset. These rare cases can confuse the model and reduce prediction accuracy. Removing them helps the model focus on normal housing patterns.

```
df = df[df['price'] < df['price'].quantile(0.99)]
df = df[df['total_sqft'] < df['total_sqft'].quantile(0.99)]
df = df[df['total_sqft'] > 100]
df = df[df['price'] > 0]
print(f"Shape after cleaning: {df.shape}")
```

10. Feature Engineering

We created new features from the existing data to give the model more useful information. For example, we extracted the number of bedrooms and calculated area per bedroom, which better represents how spacious a house is.

```
df['bhk'] = df['size'].str.split().str[0].astype(int)
```

```
df['sqft_per_bhk'] = df['total_sqft'] / df['bhk']
```

11. Creating Additional Features

We applied transformations like logarithms to reduce skewness and capture deeper relationships between variables. These transformations help the model learn patterns more effectively.

```
df['log_sqft'] = np.log1p(df['total_sqft'])
df['log_dist'] = np.log1p(df['Distance_to_nearest_MRT_station_m'])
df['bath_per_bhk'] = df['bath'] / df['bhk']
print("New features added: log_sqft, log_dist, bath_per_bhk")
```

12. Handling Location Data

Since there were many locations in the dataset, we simplified them by keeping only the most common ones and grouping the rest together. This prevents the model from becoming too complex.

```
top_locations = df['location'].value_counts().nlargest(50).index
df['location_clean'] = df['location'].apply(lambda x: x if x in top_locations else 'Other')
df = pd.get_dummies(df, columns=['location_clean'], drop_first=True)
loc_cols = [c for c in df.columns if c.startswith('location_clean_')]
print(f"Location dummy columns: {len(loc_cols)}")
```

13. Defining Features and Target

We finalized the input variables and selected price as the target variable. A log transformation was applied to price values to reduce skewness and improve prediction accuracy.

```
features = [
    'total_sqft',
    'bhk',
    'bath',
    'balcony',
    'Distance_to_nearest_MRT_station_m',
    'sqft_per_bhk',
    'log_sqft',
    'log_dist',
    'bath_per_bhk',
] + loc_cols

X = df[features]

y_log = np.log1p(df['price'])

print(f"Total features: {len(features)}")
print(f"Feature list sample: {features[:10]}")
```

14. Train–Test Split

The dataset was divided into training and testing sets. The model learns from the training data and is evaluated on the test data to check its performance on unseen data.

```
x_train, x_test, y_train_log, y_test_log = train_test_split(
    X, y_log, test_size=0.2, random_state=42
)

y_test_orig = np.expm1(y_test_log)

print(f"Train: {x_train.shape[0]} rows | Test: {x_test.shape[0]} rows")
```

15. Feature Scaling

We applied RobustScaler to make sure all features are on a similar scale. This helps the model train properly and prevents any feature from dominating the others.

```
scaler = RobustScaler()
x_train_scaled = scaler.fit_transform(x_train)
x_test_scaled  = scaler.transform(x_test)
print("Scaling done.")
```

16. Model Training

We trained a Linear Regression model using the processed data. The model learns how different features affect house prices.

```
from sklearn.linear_model import LinearRegression



model = LinearRegression()
model.fit(x_train_scaled, y_train_log)
```

17. Model Evaluation

After training, we made predictions and converted them back to the original price values so they can be understood easily.

```
y_pred_lr_log = model.predict(x_test_scaled)
y_pred_lr = np.expm1(y_pred_lr_log)
```

18. Visualization of Results

We plotted a graph to compare predicted prices with actual prices. This helped us visually check how accurate the model is.

```
plt.figure(figsize=(8, 5))
plt.scatter(y_test_orig, y_pred_lr, alpha=0.3, color='steelblue', label='Predictions')
plt.plot([y_test_orig.min(), y_test_orig.max()],
         [y_test_orig.min(), y_test_orig.max()],
         'r--', linewidth=2, label='Perfect Fit')
plt.xlabel("Actual Price (Lakhs)")
plt.ylabel("Predicted Price (Lakhs)")
plt.title("Actual vs Predicted - Housing Price (Linear Regression)")
plt.legend()
plt.show()
```

19. Affordable Housing Analysis

We classified predicted prices into affordable and expensive categories to understand housing accessibility.

```
threshold = 60
affordable = (y_pred_lr <= threshold).sum()
expensive  = (y_pred_lr > threshold).sum()

print(f"\nSDG 11 Analysis:")
print(f"Affordable units (<=60L): {affordable}")
print(f"Expensive units  (>60L):  {expensive}")
```

20. User Input Prediction

Finally, we created a system where users can enter property details and get an estimated price. This shows how the model can be used in real life.

```python
sqft     = float(input("Enter Square Footage (sqft): "))
mrt_dist = float(input("Enter Distance to MRT Station (meters): "))
bath     = int(input("Enter Number of Bathrooms: "))
balcony  = int(input("Enter Number of Balconies: "))
bhk      = int(input("Enter Number of Bedrooms (BHK): "))

sqft_per_bhk = sqft / bhk

user_data = {f: 0 for f in features}
user_data['total_sqft'] = sqft
user_data['bhk'] = bhk
user_data['bath'] = bath
user_data['balcony'] = balcony
user_data['Distance_to_nearest_MRT_station_m'] = mrt_dist
user_data['sqft_per_bhk'] = sqft_per_bhk
user_data['log_sqft'] = np.log1p(sqft)
user_data['log_dist'] = np.log1p(mrt_dist)
user_data['bath_per_bhk'] = bath / bhk

user_df = pd.DataFrame([user_data])

user_scaled = scaler.transform(user_df)

predicted_log = model.predict(user_scaled)[0]
predicted_price = np.expm1(predicted_log)

print(f"\nPredicted Price: Rs {predicted_price:.2f} Lakhs")

if predicted_price <= 60:
    print("Status: Affordable ✅")
else:
    print("Status: Expensive 🔴")
```